

Homework 3 - Nonparametric Methods, Dimensionality Reduction

Instructions

Due November 6 at 11pm

Instructions

- Use the space inside of

```
::: {.solbox data-latex=""}
```

```
:::
```

to answer the following questions.

- Do not move this file or copy it and work elsewhere. Work in the same directory.
- Use a new branch named whatever you want. Create it now! Make a small change, say by adding your name. Commit and push now!
- Try to Knit your file now. Are there errors? Fix them now, not at 11pm on the due date.
- There MUST be some text between `::: {.solbox}` and the next `:::` or this will fail to Knit.
- If your code or figures run off the edge of the `.pdf`, you'll lose 2 points automatically.
- Be sure to add your name in the `author` field at the top.

Generative AI Self Report

If you use any generative AI tool on homeworks or labs, indicate here how you used the tools and how they contributed to your learning/ability to understand the assignment. Include every prompt that you used.

This self report will not affect your grade in any way. Please be as honest as possible.

- your response here *

1 Kernel ridge regression (4 points)

```
data("arcuate")

# Split the arcuate data into training and test splits
set.seed(12345)
arcuate.train <- arcuate %>% slice_sample(n = 10, replace = FALSE)
arcuate.test <- arcuate %>% anti_join(arcuate.train, by = "position")
```

1.1 (2 points)

You will now implement kernel ridge regression on the arcuate data. The kernel you will use will be the radial basis function (RBF) kernel, which is implemented below:

```
# @param x1: A vector representing the first input x1
# @param x2: A vector representing the second input x2
# @param bw: A float representing the bandwidth for the RBF kernel
# Computes the kernel value  $k(x1, x2) = \exp(-0.5 * (x1 - x2)^2 / bw^2)$ 
rbf_kernel <- function(x1, x2, bw) {
  exp(-0.5 * (x1 - x2)^2 / bw^2)
}
```

- (1 point) Complete the function below that computes predictions on test data using kernel ridge regression with the RBF kernel.
- (1 point) Use the function you just wrote to compute predictions on the test data using a bandwidth of 30 and a regularization parameter of 0.01. Plot the predictions alongside the test and training data.

Important hints - Given matrix A and vector y , computing $A^{-1}y$ can be numerically unstable if you do it wrong. The best approach in R is to use the function `solve(A, y)`. - To avoid using a slow `for` loop to construct kernel matrices, consider using the `outer` function.

```
a.

# Computes predictions on test data using kernel ridge regression with the
# RBF kernel.
# @param x.train: A vector containing the training input
# (i.e. `arcuate.train$position`)
# @param y.train: A vector containing the training responses
# (i.e. `arcuate.train$fa`)
# @param x.test: A vector containing the test input
# (i.e. `arcuate.test$position`)
# @param bw: A float representing the bandwidth for the RBF kernel
# @param lambda: A float representing the regularization parameter
# @return A vector containing the predictions for the test data
krr <- function(x.train, y.train, x.test, bw = 0.5, lambda = 0.1) {
  # your code here
}
```

b.

1.2 (1 point)

Try adjusting with the λ parameter and plotting the predictions. Answer the following questions (no code required):

- a. (0.25 points) Describe what happens to predictions as you increase the λ parameter to very large values.
- b. (0.5 points) Using your knowledge of ridge regression, explain why this happens.
- c. (0.25 points) What happens to predictions near the training data as $\lambda \rightarrow 0$?

- a.
- b.
- c.

1.3 (1 point)

Try decreasing the bandwidth parameter and plotting the predictions. Answer the following questions (no code required):

- a. (0.5 points) Describe what happens to predictions that are far away from the training data as you decrease the bandwidth parameter. Is this behaviour desirable?
- b. (0.5 points) What is a simple transformation you can apply to your responses to make the predictions more reasonable?

- a.
- b.
- c.

2 Dimension reduction (6 points)

We will now be looking at a data set of Whisky flavour profiles. David Wishart was at one point, Chief Statistician at the Scottish Office of the Civil Service. When he retired, he focused on Whisky and wrote a book called “[Whisky Classified](#)”. For the book, he collected tasting notes published about 86 different Scotch Whisky distilleries on a number of aspects and “distilled” them down to 12 flavour categories. Then each distillery’s representative whisky was given a score on each category from 0-4, 0 meaning that that flavour is not represented in that whisky, 4 meaning that it is strongly represented.

The data set was later expanded to include more distilleries and crowd-sourced tasting notes, but, this data seems to be kept only in a for-profit Windows software which no longer exists. Dr. Wishart passed away in 2020, and there seems to be no way to access the larger data set.

You have here, the version of the data from the first edition of Dr. Wishart’s book. An article describing some of his analyses with 185 single malts was published in 2009 in [Significance](#).

We will undertake some similar analyses in this homework assignment. A snapshot of the data is shown below.

```
## tibble [86 x 16] (S3: tbl_df/tbl/data.frame)
## $ Distillery: chr [1:86] "Aberfeldy" "Aberlour" "AnCnoc" "Ardbeg" ...
## $ Body      : num [1:86] 2 3 1 4 2 2 0 2 2 2 ...
## $ Sweetness : num [1:86] 2 3 3 1 2 3 2 3 2 3 ...
## $ Smoky     : num [1:86] 2 1 2 4 2 1 0 1 1 2 ...
## $ Medicinal : num [1:86] 0 0 0 4 0 1 0 0 0 1 ...
## $ Tobacco   : num [1:86] 0 0 0 0 0 0 0 0 0 0 ...
## $ Honey     : num [1:86] 2 4 2 0 1 1 1 2 1 0 ...
## $ Spicy     : num [1:86] 1 3 0 2 1 1 1 1 0 2 ...
## $ Winey     : num [1:86] 2 2 0 0 1 1 0 2 0 0 ...
## $ Nutty     : num [1:86] 2 2 2 1 2 0 2 2 2 2 ...
## $ Malty     : num [1:86] 2 3 2 2 3 1 2 2 2 1 ...
## $ Fruity    : num [1:86] 2 3 3 1 1 1 3 2 2 2 ...
## $ Floral    : num [1:86] 2 2 2 0 1 2 3 1 2 1 ...
## $ Postcode  : chr [1:86] "PH15 2EB" "AB38 9PJ" "AB5 5LI" "PA42 7EB" ...
## $ Longitude : num [1:86] -3.85 -3.23 -2.79 -6.11 -2.74 ...
## $ Latitude  : num [1:86] 56.6 57.5 57.4 55.6 57.4 ...
```

2.1 (0.5 points)

Create a bar chart that shows 12 panels, one for each of the twelve flavour profiles. In each panel, the heights of the bars should be the percent of whiskys with each score 0-4 (out of 86 total) (the y-axis is a percent between 0 and 100). Make sure that your graphic fits on 1 page.

```
# your code
```

2.2 (0.5 points)

Charts should always be described in any analysis or else they are useless. Examining your chart, describe any patterns you see in a few sentences (at most 3). There are many right answers. Wrong answers are things like “there are 12 panels” or “each panel has 5 bars” or “I don’t see anything”.

Your answer

2.3 (1 point)

Perform PCA on the 12 flavour categories. Produce a scree plot with the y-axis displaying the percent of total variance explained by each component. Based on your plot, is 2 a reasonable number of dimensions to use? Why or why not?

```
# your code
```

Your text

2.4 (1 point)

Produce a plot of PC1 vs PC2. Instead of a dot for each distillery, show the name of the distillery in the figure. (You'll have lots of words, some of which may be hard to read). Here's example code to get the idea.

```
plot(1:5,1:5, pch = "")  
text(1:5,1:5, labels = letters[1:5])
```

In ggplot, you would use `geom_text()` or `geom_label()` rather than `geom_point()`.

Also produce a plot of the Loadings for the first two PCs. Which qualities seem important (positive or negative) for each PC (describe this in 1-2 sentences)? Do any Distilleries stick out, if so, what qualities do they have in common?

```
# your code to plot pc1 against pc2
```

```
# your code to plot the loadings
```

Your text

2.5 (2 points)

Complete the function below. See the documentation for help with the arguments. Your function must return a matrix of dimension $n \times M$ where n is the number of rows of K . Do not change the function signature.

You should make your function produce errors or warnings if invalid inputs are passed. This is easiest way to do with `stop()` or `stopifnot()` or `warning()`. Try examining the documentation for those functions. Looking at the tests should help you determine what sorts of checks to perform and what warnings to throw.

Hint: think carefully about what the sign of `tol` should be to make this work. It may not conform to your first instinct.

```
##' @param K a symmetric, non-negative definite Kernel matrix
##' @param M positive integer for the target embedding dimension
##' @param tol tolerance to test for negative definiteness.
kpca <- function(K, M = 2, tol = -1e-8) {
  # your code
}
```

2.6 (1 point)

Use your `kpca()` function to estimate kernel PCA. Form your `K` by first calculating the distance matrix between rows in your data using the `canberra` distance. The easiest way is with the `dist()` function. You have to convert the result to a matrix with `as.matrix()`. Then set `K` to be `1 - this result`. Pass that in to your function. Use `M = 2`.

Now the hard part, produce a 12 panel plot as in 1. The x-axis should be the first component, the y-axis should be the second component. Colour the points based on the flavour score (0-4). Describe any patterns you notice in 2-3 sentences.

```
# your code
```

Your text